



SIMATS Online Programming Lab

JAVA PROGRAMMING



Sagala Venkata Anirudh
192572110RB

CO3-AT3-Assignment

[← Back](#)

QUESTIONS

Q1

Bank Account ♦ Concurrent
Deposit and Withdrawal
25 marks

**Q2**

Traffic Signal Simulation ♦
Thread Control
25 marks

**Q3**

Thread Priority ♦ Hospital
Emergency Tasks
25 marks

**Q4**

Online Login System ♦
Multiple Exceptions
25 marks



4/4 answered

Q1

Bank Account ♦ Concurrent Deposit and Withdrawal

25
marks

Problem Statement:

Develop a Java program in which multiple threads perform deposit and withdrawal operations on the same bank account.

Requirements:

Create deposit and withdrawal threads.

Maintain a shared account balance.

Use the synchronized mechanism.

Prevent inconsistent balance updates.

Display transaction details.

Your Code

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         final int[] balance = {sc.nextInt()};
6         int d1 = sc.nextInt();
7         int d2 = sc.nextInt();
8         int w1 = sc.nextInt();
9         int w2 = sc.nextInt();
10        Runnable deposit1 = new Runnable() {
```

```
11         public void run() {
12             synchronized (balance) {
13                 balance[0] += d1;
14                 System.out.println("Depo
15             }
16         }
17     };
18     Runnable deposit2 = new Runnable() {
19         public void run() {
20             synchronized (balance) {
21                 balance[0] += d2;
22                 System.out.println("Depo
23             }
24         }
25     };
26     Runnable withdraw1 = new Runnable()
27         public void run() {
28             synchronized (balance) {
29                 if (w1 <= balance[0]) {
30                     balance[0] -= w1;
31                     System.out.println("
32                 } else {
33                     System.out.println("
34                 }
35             }
36         }
37     };
38     Runnable withdraw2 = new Runnable()
39         public void run() {
40             synchronized (balance) {
41                 if (w2 <= balance[0]) {
42                     balance[0] -= w2;
43                     System.out.println("
44                 } else {
45                     System.out.println("
46                 }
47             }
48         }
49     };
50     Thread t1 = new Thread(deposit1);
51     Thread t2 = new Thread(deposit2);
52     Thread t3 = new Thread(withdraw1);
53     Thread t4 = new Thread(withdraw2);
54     t1.start();
55     t2.start();
```

```
56         t3.start();
57         t4.start();
58         t1.join();
59         t2.join();
60         t3.join();
61         t4.join();
62         System.out.println("Final Balance: ")
63     }
64 }
```



Input

1000
500
300
200



Output



Run



Save